Searching

Content notes, classwork and homework with worked C++ solutions.

Lesson 1: Linear Search

Quick recap: Finding elements in an unsorted array • Count occurrences • First and last index

Theory & Nuances

Linear search

Linear search checks the elements one by one from index 0 until it finds the target x. If it finds x at index i it returns i straight away; if it reaches the end without finding x it returns -1, a value that can never be a real index. It works on ANY array, sorted or not, and is the only option when the data is unsorted.

Time complexity

Best case: x is the first element, 1 comparison, O(1). Worst case: x is last or missing, n comparisons, O(n). On average about n / 2 comparisons, which is still O(n). Doubling the array doubles the work. For many searches on the same big array it pays to sort it once and use binary search.

Count occurrences

To count how many times x appears you cannot stop at the first match - walk the whole array and add 1 for every element equal to x. This always takes n comparisons. The same pattern answers "does x exist?" (count > 0), but a dedicated check that returns true at the first match is faster on average.

First and last index

The first index is the first match seen while walking left to right: remember it only if it has not been set yet (first == -1). The last index is the latest match seen, so overwrite it at every match. One pass gives both. Alternatively, search from the right end (i = n - 1 down to 0) and stop at the first match to get the last index.

Returning a bool or an index

Write the search as a function. If the caller only needs yes/no, return a bool: return true; inside the loop, return false; after it. If the caller needs the position, return the index, or -1 for not found. The index of the maximum works the same way: keep the index of the best value seen so far.

Key Points / Common Mistakes

- Return -1 (not 0) for "not found" - 0 is a real index.
- Put return -1 / return false AFTER the loop, not in an else inside it, or you stop after one element.
- Counting needs the whole array; finding can stop at the first match.
- First index: set only once (when first == -1). Last index: overwrite at every match.
- For the index of the max, use > to keep the FIRST maximum and >= to keep the LAST one.
- Linear search is O(n) and works on unsorted data; binary search needs sorted data.

Classwork Problems

1. Linear search - return the index

```
#include <iostream>
#include <vector>
using namespace std;

int linearSearch(const vector<int> &arr, int x) {
    for (int i = 0; i < (int)arr.size(); i++) {
        if (arr[i] == x) {
            return i;                   // found: stop right here
        }
    }
    return -1;                          // checked everything: not found
}

int main() {
    vector<int> arr = {14, 3, 27, 9, 41, 6};

    cout << "Index of 9  = " << linearSearch(arr, 9) << endl;
    cout << "Index of 14 = " << linearSearch(arr, 14) << endl;
    cout << "Index of 50 = " << linearSearch(arr, 50) << " (not found)" << endl;
    return 0;
}
```

Output

```
Index of 9  = 3
Index of 14 = 0
Index of 50 = -1 (not found)
```

2. Count occurrences of x

```
#include <iostream>
#include <vector>
using namespace std;

int countOccurrences(const vector<int> &arr, int x) {
    int count = 0;
    for (int value : arr) {
        if (value == x) {
            count++;                    // do not stop: keep counting
        }
    }
    return count;
}

int main() {
    vector<int> arr = {2, 5, 2, 8, 2, 5, 1};

    cout << "2 appears " << countOccurrences(arr, 2) << " times" << endl;
    cout << "5 appears " << countOccurrences(arr, 5) << " times" << endl;
    cout << "7 appears " << countOccurrences(arr, 7) << " times" << endl;
    return 0;
}
```

Output

```
2 appears 3 times
5 appears 2 times
7 appears 0 times
```

3. First and last index of x

```
#include <iostream>
#include <vector>
using namespace std;

void firstAndLast(const vector<int> &arr, int x) {
    int first = -1, last = -1;
    for (int i = 0; i < (int)arr.size(); i++) {
        if (arr[i] == x) {
            if (first == -1) {
                first = i;              // only the first match
            }
            last = i;                   // every match moves last
        }
    }
    cout << "x = " << x << ": first = " << first << ", last = " << last << endl;
}

int main() {
    vector<int> arr = {4, 7, 1, 7, 9, 7, 2};
    firstAndLast(arr, 7);
    firstAndLast(arr, 4);
    firstAndLast(arr, 3);
    return 0;
}
```

Output

```
x = 7: first = 1, last = 5
x = 4: first = 0, last = 0
x = 3: first = -1, last = -1
```

Homework Problems

1. Check if a value exists (boolean)

```
#include <iostream>
#include <vector>
using namespace std;

bool exists(const vector<int> &arr, int x) {
    for (int value : arr) {
        if (value == x) {
            return true;
        }
    }
    return false;                       // after the loop, not inside it
}

int main() {
    vector<int> arr = {11, 23, 5, 42, 17};
    int queries[] = {42, 8, 11};

    for (int q : queries) {
        if (exists(arr, q)) {
            cout << q << " exists" << endl;
        } else {
            cout << q << " does not exist" << endl;
        }
    }
    return 0;
}
```

Output

```
42 exists
8 does not exist
11 exists
```

2. Return index of the maximum element

```
#include <iostream>
#include <vector>
using namespace std;

int indexOfMax(const vector<int> &arr) {
    int best = 0;                       // index of the biggest so far
    for (int i = 1; i < (int)arr.size(); i++) {
        if (arr[i] > arr[best]) {       // > keeps the FIRST maximum
            best = i;
        }
    }
    return best;
}

int main() {
    vector<int> arr = {23, 67, 12, 67, 45};
    int idx = indexOfMax(arr);
    cout << "Maximum " << arr[idx] << " is at index " << idx << endl;
    return 0;
}
```

Output

```
Maximum 67 is at index 1
```

Lesson 2: Binary Search

Quick recap: Searching a sorted array in O(log n) • lower_bound / upper_bound basics • Floor and ceil

Theory & Nuances

The idea

Binary search only works on a SORTED array. Look at the middle element: if it is the target, done. If the target is bigger, it can only be in the right half, so throw away the left half (and the middle). If it is smaller, throw away the right half. Every step halves the search space, like finding a word in a dictionary by opening it in the middle.

The loop

Keep two indexes, low = 0 and high = n - 1, for the part that may still contain the target. While low <= high: compute mid = low + (high - low) / 2, then move low = mid + 1 or high = mid - 1. If the loop ends (low > high) the target is not there. Writing low + (high - low) / 2 instead of (low + high) / 2 avoids int overflow when the indexes are large.

Why O(log n)

After k steps only n / 2^k elements are left, so the search ends after about log2(n) steps. A sorted array of 1,000,000 elements needs at most 20 comparisons, where linear search may need 1,000,000. The price is that the array must be sorted first (O(n log n) once), which pays off when you search it many times.

First occurrence, search insert position

With duplicates, plain binary search returns ANY matching index. For the first occurrence, when arr[mid] == x save ans = mid and keep searching LEFT (high = mid - 1); for the last occurrence, save and go RIGHT. When the loop ends without finding x, low is exactly where x would be inserted to keep the array sorted - that is LeetCode 35 Search Insert Position.

lower_bound / upper_bound

The <algorithm> header does this for you. lower_bound(v.begin(), v.end(), x) points to the first element >= x, and upper_bound(v.begin(), v.end(), x) to the first element > x. Subtract v.begin() to get an index. So the first index of x is lower_bound (if v[i] == x), the last index is upper_bound - 1, and the count of x is upper_bound - lower_bound. If nothing qualifies, you get v.end(), i.e. index n.

Floor and ceil

The ceil of x is the smallest element >= x (exactly what lower_bound finds). The floor of x is the largest element <= x. With manual binary search: for ceil, whenever arr[mid] >= x save it and go left; for floor, whenever arr[mid] <= x save it and go right. If nothing qualifies, report "no ceil" / "no floor".

Key Points / Common Mistakes

- Binary search only works on sorted data - sort first, or use linear search.
- Loop while low <= high (not <), or you skip the last remaining element.
- Always move past mid: low = mid + 1 and high = mid - 1, otherwise the loop can run forever.
- Use mid = low + (high - low) / 2 to avoid overflow.
- lower_bound = first >= x, upper_bound = first > x; subtract v.begin() for the index.
- If x is missing, low ends at the insert position; check the bounds before reading arr[low].

Classwork Problems

1. LC704 Binary Search

```
#include <iostream>
#include <vector>
using namespace std;

int search(const vector<int> &nums, int target) {
    int low = 0, high = nums.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        cout << "  low = " << low << ", high = " << high << ", mid = " << mid
             << " (" << nums[mid] << ")" << endl;
        if (nums[mid] == target) {
            return mid;
        } else if (nums[mid] < target) {
            low = mid + 1;              // target is in the right half
        } else {
            high = mid - 1;             // target is in the left half
        }
    }
    return -1;
}

int main() {
    vector<int> nums = {-1, 0, 3, 5, 9, 12};

    cout << "Search 9:" << endl;
    int idx = search(nums, 9);
    cout << "Index = " << idx << endl;

    cout << "Search 2:" << endl;
    idx = search(nums, 2);
    cout << "Index = " << idx << " (not found)" << endl;
    return 0;
}
```

Output

```
Search 9:
  low = 0, high = 5, mid = 2 (3)
  low = 3, high = 5, mid = 4 (9)
Index = 4
Search 2:
  low = 0, high = 5, mid = 2 (3)
  low = 0, high = 1, mid = 0 (-1)
  low = 1, high = 1, mid = 1 (0)
Index = -1 (not found)
```

2. LC35 Search Insert Position

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int searchInsert(const vector<int> &nums, int target) {
    int low = 0, high = nums.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (nums[mid] == target) {
            return mid;
        } else if (nums[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return low;                         // where target would be inserted
}

int main() {
    vector<int> nums = {1, 3, 5, 6};
    int targets[] = {5, 2, 7, 0};

    for (int t : targets) {
        int pos = searchInsert(nums, t);
        int lb = lower_bound(nums.begin(), nums.end(), t) - nums.begin();
        cout << "target " << t << " -> " << pos << "  (lower_bound gives " << lb << ")" << endl;
    }
    return 0;
}
```

Output

```
target 5 -> 2  (lower_bound gives 2)
target 2 -> 1  (lower_bound gives 1)
target 7 -> 4  (lower_bound gives 4)
target 0 -> 0  (lower_bound gives 0)
```

3. First occurrence using binary search

```
#include <iostream>
#include <vector>
using namespace std;

int firstOccurrence(const vector<int> &arr, int x) {
    int low = 0, high = arr.size() - 1;
    int ans = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == x) {
            ans = mid;                 // a match - but maybe not the first
            high = mid - 1;            // keep looking on the left
        } else if (arr[mid] < x) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return ans;
}

int main() {
    vector<int> arr = {1, 2, 2, 2, 3, 4, 4, 5};

    cout << "First 2 at index " << firstOccurrence(arr, 2) << endl;
    cout << "First 4 at index " << firstOccurrence(arr, 4) << endl;
    cout << "First 6 at index " << firstOccurrence(arr, 6) << endl;
    return 0;
}
```

Output

```
First 2 at index 1
First 4 at index 5
First 6 at index -1
```

Homework Problems

1. LC34 First and Last Position of Element

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

// goLeft = true finds the first position, false finds the last
int findBound(const vector<int> &nums, int target, bool goLeft) {
    int low = 0, high = nums.size() - 1;
    int ans = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (nums[mid] == target) {
            ans = mid;
            if (goLeft) {
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        } else if (nums[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return ans;
}

int main() {
    vector<int> nums = {5, 7, 7, 8, 8, 10};
    int targets[] = {8, 7, 6};

    for (int t : targets) {
        int first = findBound(nums, t, true);
        int last = findBound(nums, t, false);
        cout << "target " << t << " -> [" << first << ", " << last << "]" << endl;
    }

    // the same with the STL
    int lo = lower_bound(nums.begin(), nums.end(), 8) - nums.begin();
    int hi = upper_bound(nums.begin(), nums.end(), 8) - nums.begin();
    cout << "STL for 8: first = " << lo << ", last = " << hi - 1
         << ", count = " << hi - lo << endl;
    return 0;
}
```

Output

```
target 8 -> [3, 4]
target 7 -> [1, 2]
target 6 -> [-1, -1]
STL for 8: first = 3, last = 4, count = 2
```

2. Find Ceil of x in sorted array

```cpp
#include <iostream>
#include <vector>
using namespace std;

// index of the smallest element >= x, or -1 if there is none
int ceilIndex(const vector<int> &arr, int x) {
    int low = 0, high = arr.size() - 1;
    int ans = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] >= x) {
            ans = mid;                  // a candidate; look for a smaller one
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }
    return ans;
}

int main() {
    vector<int> arr = {1, 2, 8, 10, 10, 12, 19};
    int queries[] = {5, 10, 0, 20};

    for (int x : queries) {
        int idx = ceilIndex(arr, x);
        if (idx == -1) {
            cout << "ceil(" << x << ") does not exist" << endl;
        } else {
            cout << "ceil(" << x << ") = " << arr[idx] << " at index " << idx << endl;
        }
    }
    return 0;
}
```

Output

```
ceil(5) = 8 at index 2
ceil(10) = 10 at index 3
ceil(0) = 1 at index 0
ceil(20) does not exist
```